

TaxaFetch

Taxonomic assignments are more accurate when they consider which species are plausible at the sampling location. Without this context, classifiers frequently assign detections to ecologically implausible taxa. TaxaFetch compiles species occurrence records from multiple sources – GBIF (Global Biodiversity Information Facility; GBIF Secretariat, Copenhagen, Denmark), DataONE (Data Observation Network for Earth; University of New Mexico, Albuquerque, New Mexico), BioTIME (database maintained by the University of St Andrews, St Andrews, Scotland, United Kingdom), and published literature – and combines them into a standardized format for downstream habitat assignment (TaxaHabitat) and prior estimation (TaxaExpect).

Data Sources

Source	Function	What it provides
GBIF	<code>fetch_gbif_occurrences()</code>	Global occurrence records via download API
DataONE	<code>fetch_dataone_occurrences()</code>	Ecological datasets from DataONE repositories
BioTIME	<code>read_biotime_study()</code>	Time-series biodiversity data
Literature	<code>search_literature()</code>	OpenAlex (operated by OurResearch, a nonprofit organization) scholarly search + PDF download
PDFs	<code>extract_pdf_text()</code>	Extract occurrence data from published PDFs
iNaturalist	<code>fetch_inat_occurrences()</code> , <code>check_inat_range()</code>	Local observation counts and known-range checks via the iNaturalist API

Installation

```
# Requires TaxaTools (foundation package)
devtools::install("path/to/TaxaTools")
devtools::install("path/to/TaxaFetch")
```

Quick Start

```
library(TaxaFetch)

# 1. Define a bounding box (Southern California coast)
bbox <- make_bbox_wkt(lat = 34.0, lon = -119.25, radius_deg = 0.5)

# 2. Get GBIF taxon keys from verified names, with full hierarchy context
#    to avoid homonym errors
hierarchy_df <- data.frame(
  genus   = c("Fundulus", "Atherinops"),
  species = c("Fundulus parvipinnis", "Atherinops affinis")
)
```

```

keys <- get_keys_from_context(hierarchy_df)

# 3. Fetch GBIF occurrences
gbif_data <- fetch_gbif_occurrences(keys = keys$usageKey, geometry = bbox)

# 4. Filter to high-quality records
filtered <- filter_gbif_quality(gbif_data, max_coord_uncertainty = 500)

# 5. Stack multiple sources into one table
all_data <- stack_occurrences(gbif = filtered)

```

Key Functions

GBIF pipeline: - `make_bbox_wkt()` – create WKT bounding box for spatial queries - `get_keys_from_context()` – resolve taxon names to GBIF usage keys - `fetch_gbif_occurrences()` – download occurrence records - `filter_gbif_quality()` – remove low-quality records

DataONE pipeline: - `search_dataone()` – find datasets by taxon and location - `harvest_dataone_catalog()` – build a dataset catalog with LLM screening - `fetch_dataone_occurrences()` – download and standardize occurrence tables

Literature + PDF pipeline: - `search_literature()` – OpenAlex scholarly search - `download_literature_pdfs()` – batch PDF download - `extract_pdf_text()` – section-aware text extraction - `screen_pdf_structure()` – LLM-based relevance screening - `build_pdf_extract_prompt()` / `parse_pdf_extract_response()` – LLM data extraction

Combining sources: - `stack_occurrences()` – row-bind sources into a single standardized table - `report_fetch()` – summarize data acquisition for reports

API Keys

TaxaFetch uses GBIF credentials for occurrence downloads and OpenAlex for literature search. See the TaxaTools API Setup vignette for configuration.

Cache

`download_gbif_occurrences()`, `fetch_gbif_occurrences()`, and `check_geographic_outliers()` all cache to a persistent, per-user directory (`tools::R_user_dir("TaxaFetch", "cache")`) so re-running the same query skips the GBIF wait. GBIF download zips in particular can be multi-gigabyte and are cached **permanently, with no automatic expiration** – re-running a query with `overwrite = TRUE` replaces the cached zip (asking for confirmation first in an interactive session) but otherwise nothing is ever cleared for you. If GBIF itself refuses the download request (a transient 5xx), the request is retried with backoff, and with `overwrite = TRUE` a verified cached zip for the identical query is used instead – with a warning, never silently – so a long run does not die at the fetch step (`on_submit_failure = "error"` restores the old behaviour).

Run `taxafetch_clear_cache(dry_run = TRUE)` to see how much space the cache is using before clearing it, or `taxafetch_clear_cache()` to clear it directly. `download_gbif_occurrences()` also reports the cache's total size after every run and offers to clear it once it passes 1 GB.

Vignettes

- Data Acquisition – end-to-end guide

Part of TaxaID

TaxaFetch is the data acquisition layer in the TaxaID ecosystem. Its output feeds into TaxaHabitat (habitat assignment) and TaxaExpect (prior estimation).

Ecosystem: TaxaTools -> **TaxaFetch** -> TaxaHabitat -> TaxaExpect -> TaxaAssign

See the TaxaID README for ecosystem overview and installation instructions.

Citation

Lafferty, K.D., 2026, TaxaID – A modular R ecosystem for Bayesian taxonomic assignment: U.S. Geological Survey software release, <https://doi.org/10.5066/xxxxxx>.

Software Requirements

- R ($\geq 4.1.0$; R Core Team 2025)
- TaxaTools (foundation package, installed first)
- rgbif (Chamberlain et al. 2025; for GBIF occurrence queries)
- An Application Programming Interface (API) key is needed for literature screening and PDF extraction (Anthropic Claude, Google Gemini, OpenAI, or local Ollama – see TaxaTools)

All dependencies are declared in the DESCRIPTION file and installed automatically.

Developed with Claude Code (Anthropic PBC, San Francisco, California).

References

Chamberlain, S., Barve, V., McGlinn, D., Oldoni, D., Desmet, P., Geffert, L. and Ram, K. (2025). rgbif: Interface to the Global Biodiversity Information Facility API. R package. <https://CRAN.R-project.org/package=rgbif>

R Core Team (2025). R: A Language and Environment for Statistical Computing. V.4.5.2. R Foundation for Statistical Computing, Vienna, Austria. <https://www.r-project.org>